How Routing Works in ASP.NET Core Web API

Back to: ASP.NET Core Web API Tutorials

How Routing Works in ASP.NET Core Web API

Routing is the **process of mapping incoming HTTP requests to the appropriate application logic**, such as a controller action, a Razor Page, or a Minimal API endpoint. In ASP.NET Core, this process is powered by the **Endpoint Routing System**, which acts like a smart traffic controller that decides where each request should go and how it should be processed.

Before understanding how Routing Works in ASP.NET Core Web API, let us first understand the core components where the Routing Information is stored.

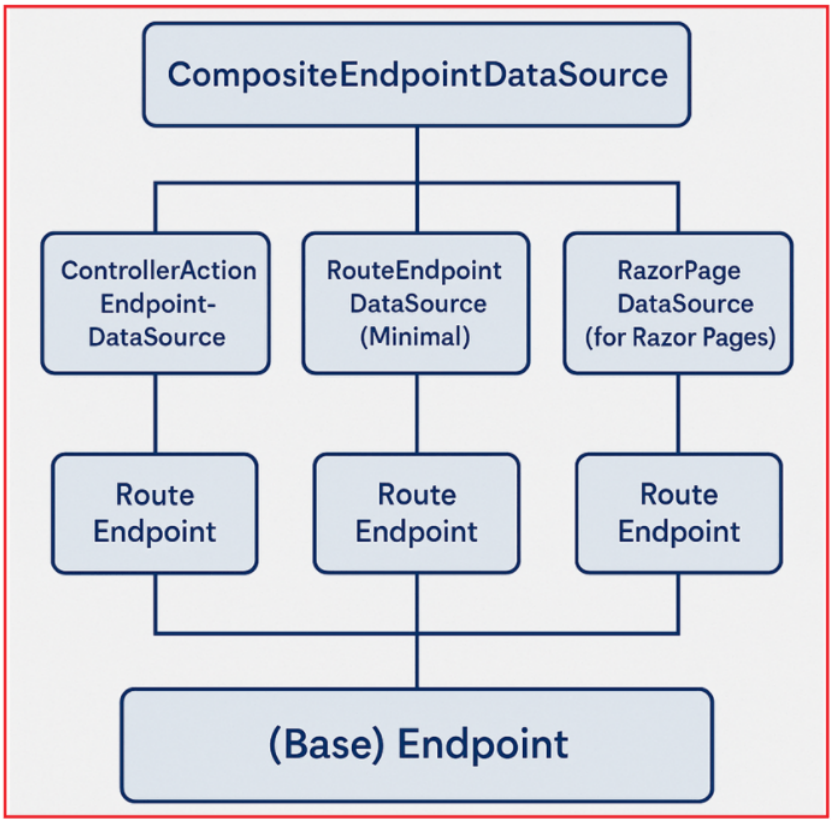
Understanding Core Routing Components in ASP.NET Core:

Routing is one of the most important internal mechanisms of ASP.NET Core. It decides **which piece of code** (controller action, Razor Page, Minimal API, etc.) should handle a given HTTP request. To make this work efficiently, ASP.NET Core uses a few key internal classes:

- **Endpoint**
- **RouteEndpoint**
- **EndpointDataSource**
- **CompositeEndpointDataSource**

Together, they form the backbone of the **Endpoint Routing System**, the engine that connects incoming URLs to the correct request handlers. For a better understanding, please have a look at the following image, which shows the high-level overview of these classes and their relationships.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Let's go step-by-step and understand **how these classes work together internally**, what role each one plays, and how the routing system uses them during a request.

The Endpoint Class:

At the heart of the Endpoint Routing System is the **Endpoint** class. Think of it as the **Universal Contract** that represents *anything* in ASP.NET Core capable of handling a request, whether that's a Controller Action, a Razor Page, a Minimal API, or even a SignalR Hub. The key idea is that the routing middleware doesn't care *what* it's routing to; it only cares *that* it's routing to an Endpoint.

What an Endpoint Contains

Each Endpoint object holds three main pieces of information:

1. **RequestDelegate**: A compiled function (delegate) that executes when this endpoint is matched. It's essentially *the method that handles the request*.
2. **Metadata Collection**: A list of attributes and behaviors attached to this endpoint, such as [Authorize], [HttpGet], [Produces], or custom metadata used by middleware.
3. **DisplayName**: A human-readable identifier used for debugging, logging, or displaying in tools like Swagger.

This abstraction allows ASP.NET Core to treat all kinds of request handlers uniformly. Controllers, Razor Pages, Minimal APIs, SignalR hubs, static files, and other request targets are all represented internally as **Endpoints**.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

The RouteEndpoint Class:

While Endpoint is a general concept, many web applications (MVC, Web API) depend on **Route Templates**, for example, **/api/products/{id}**. To represent such endpoints, ASP.NET Core provides a specialized subclass called **RouteEndpoint**.

What RouteEndpoint Adds

RouteEndpoint extends **Endpoint** and adds **Routing-Specific Data,** such as:

- **RoutePattern**: Represents the parsed structure of a Route Template (like “api/products/{id?}”), including Parameters, Constraints, and Default Values.
- **Order**: Defines the priority used to resolve conflicts when multiple routes could match the same URL.
- **Endpoint Data:** Everything inherited from Endpoint (RequestDelegate, Metadata, DisplayName).

Why RouteEndpoint Exists

Not all endpoints in ASP.NET Core follow a route template:

- SignalR hubs connect via a fixed hub path, not a route.
- Static file middleware and Swagger UI endpoints don’t use {controller}/{action} patterns.

The **RouteEndpoint** is used only for endpoints that define route templates, while other endpoints remain as lightweight **Endpoint** objects. Every controller action, Razor Page, or Minimal API that includes a route pattern is represented as a **RouteEndpoint** and stored in memory. This separation ensures that the routing system remains both **flexible** and **efficient**.

The EndpointDataSource Class:

The **EndpointDataSource** acts as the central provider of all endpoints that the ASP.NET Core routing system can recognize. It’s like a factory or a repository that “produces” and manages the collection of available endpoints for a particular subsystem.

Each feature in ASP.NET Core that defines routes, such as **MVC controllers**, **Razor Pages**, **Minimal APIs**, or **SignalR hubs,** registers its own dedicated **EndpointDataSource** during application startup. The framework then combines these data sources into a single, unified list that the **Routing Middleware** uses to match incoming requests.

In simpler terms, the EndpointDataSource is the **bridge between route discovery and route matching**. It gathers all possible endpoints during startup and makes them available in memory so that, when a request arrives, the routing system can efficiently look up and select the correct endpoint to handle it.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Examples

- When you call **MapControllers()**, ASP.NET Core creates and registers a **ControllerActionEndpointDataSource**, which contains all endpoints representing your controller actions.
- When you call **MapRazorPages()**, a **PageActionEndpointDataSource** is registered, holding all the endpoints that correspond to your Razor Pages.
- When you define **Minimal APIs** using methods such as **MapGet()**, **app.MapPost()**, or **app.MapDelete()**, ASP.NET Core adds a **RouteEndpointDataSource** that stores these lightweight route definitions.
- Similarly, when you enable **SignalR** with **MapHub<T>()**, the framework registers a **HubEndpointDataSource**, which contains endpoints for SignalR hubs and manages real-time communication routes.

The CompositeEndpointDataSource Class

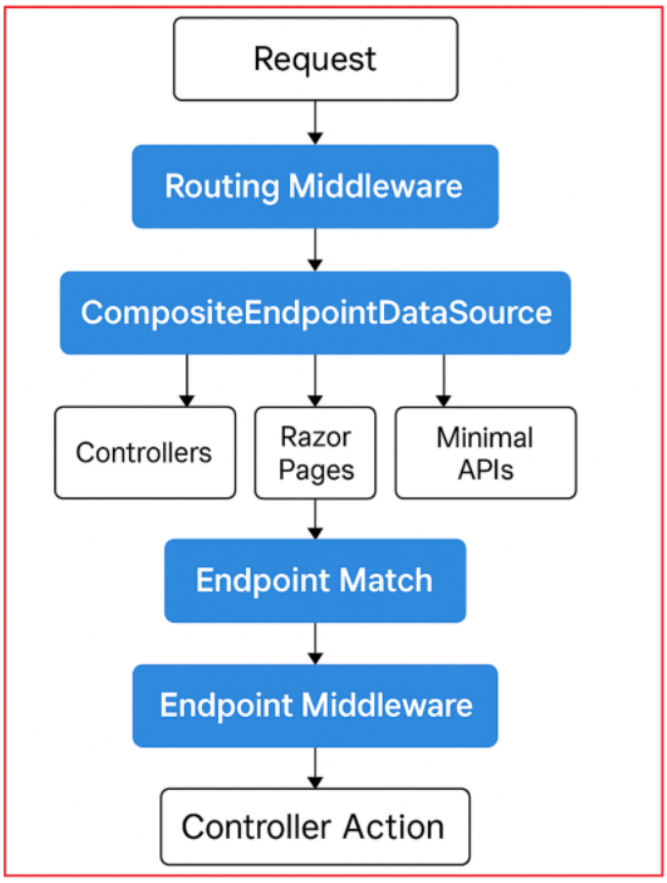
Since multiple data sources exist (controllers, Razor Pages, Minimal APIs, etc.), the routing system needs a single unified source to work with. That's the job of the **CompositeEndpointDataSource**. The framework combines all individual data sources, controllers, Razor Pages, minimal APIs, SignalR hubs, and static files into a unified collection called **CompositeEndpointDataSource.**

When the app starts:

1. Each subsystem (MVC, Razor Pages, Minimal APIs) registers its own EndpointDataSource.
2. ASP.NET Core Framework automatically creates a single CompositeEndpointDataSource that aggregates them.
3. The routing middleware queries this **Single Composite Endpoint DataSource** when matching requests.

So, when a request like /api/products/5 arrives, the routing system doesn't check each subsystem separately; it checks this composite, which internally looks across all registered data sources. This ensures that **no matter how an endpoint is defined**, it can always be found and executed through one consistent routing pipeline. For a better understanding, please have a look at the following diagram.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



How Routing Works in ASP.NET Core Web API:

To truly understand How Routing Works in ASP.NET Core Web API internally, let's walk step-by-step from **Application Startup** to **Request Execution**, exploring how ASP.NET Core builds, stores, and executes routes under the hood.

Step 1: Application Startup Begins

When an ASP.NET Core application starts, the **Program.cs** file is the first to execute. It sets up the application host, configures services (like MVC), builds the middleware pipeline, and ultimately starts the web server. The following is the basic structure of a Web API program:

```
namespace RoutingInASPNETCoreWebAPI
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            // Add services to the container.
            builder.Services.AddControllers();
            builder.Services.AddEndpointsApiExplorer();
            builder.Services.AddSwaggerGen();
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
            var app = builder.Build();

            // Configure the HTTP request pipeline.
            if (app.Environment.IsDevelopment())
            {
                app.UseSwagger();
                app.UseSwaggerUI();
            }

            app.UseHttpsRedirection();

            app.UseAuthorization();

            app.MapControllers();

            app.Run();
        }
    }
}
```

Why does the application work even though there's no explicit app.UseRouting()?

Starting from **.NET 6**, ASP.NET Core introduced the **Minimal Hosting Model**, where routing is automatically configured internally. When you call methods like:

- app.MapControllers();
- app.MapGet("/weather", …);

The framework **Implicitly Adds** both the **Routing Middleware** and **Endpoint Middleware** in the correct order. In older versions (like .NET 3.1 or .NET 5), you had to explicitly write:

- app.UseRouting();
- app.UseEndpoints(endpoints => endpoints.MapControllers());

Now, app.MapControllers() **Automatically Handles** both steps, simplifying startup configuration.

Step 2: AddControllers() Registers MVC Services

When we call the builder.Services.AddControllers(), ASP.NET Core registers all internal MVC services required for Web API functionality. These include:

- **Controller Discovery and Activation**
- **Model Binding and Validation**

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- **Action and Result Filters**
- **Routing Metadata Providers**

At this stage, **No Actual Routes Are Created Yet**. The system has only registered the services required to discover controllers and build endpoints later. Think of it as **gathering all the ingredients before cooking**; everything is ready, but no dish has been made yet.

Step 3: Build() Finalizes the Service Container

When we call the **builder.Build(),** ASP.NET Core finalizes the **Dependency Injection (DI)** container. This step locks in all registered services, including routing, controller discovery, and metadata infrastructure, so they're ready for use when the middleware pipeline is built.

Still, **No Routes Exist in Memory Yet**; the application only *knows how* to build them later.

Step 4: UseRouting() Adds the Routing Middleware (Implicitly)

In .NET 6+, this step is handled automatically. The **Routing Middleware** is what reads incoming request URLs and decides *which endpoint* should handle them. When this middleware runs, it:

1. Reads the request URL.
2. Matches it against known route templates.
3. Stores the matched endpoint inside HttpContext.

However, at the application startup, it only **registers** itself in the pipeline. When an incoming HTTP Request comes, it will perform the above functions. Not at the time of application startup.

Step 5: MapControllers(), Endpoint Creation

In case of ASP.NET Core Web API Application, when the app.MapControllers() executes:

1. ASP.NET Core scans all assemblies for classes ending in "Controller".
2. It reads [Route], [HttpGet], [HttpPost], etc. attributes.
3. It constructs **RouteEndpoint** objects for each action method.

Each **Route Endpoint** includes:

- A **Route Pattern** (e.g., /api/products/{id}).
- A **RequestDelegate** (a compiled function that knows how to execute that controller action).
- A **Metadata Collection** (e.g., [Authorize], [HttpGet], custom filters).

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

These endpoints are then stored in memory inside the EndpointDataSource. From this point forward, **routing never rebuilds these objects**; they're created **once at startup** and reused for every request. For example, please add the following ProductsController**.**

```
using Microsoft.AspNetCore.Mvc;
namespace RoutingInASPNETCoreWebAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ProductsController : ControllerBase
    {
        private static readonly List<Product> _products = new()
        {
            new Product { Id = 1, Name = "Laptop", Price = 55000 },
            new Product { Id = 2, Name = "Keyboard", Price = 1200 },
            new Product { Id = 3, Name = "Mouse", Price = 800 }
        };

        [HttpGet]
        public IActionResult GetAll()
        {
            return Ok(_products);
        }
        [HttpGet("{id}")]
        public IActionResult GetById(int id)
        {
            var p = _products.FirstOrDefault(x => x.Id == id);
            if (p == null) return NotFound($"Product {id} not found.");
            return Ok(p);
        }
    }

    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; } = null!;
        public decimal Price { get; set; }
    }
}
```

The Discovered Endpoints

When the above controller is discovered during the app.MapControllers(), ASP.NET Core builds the following **RouteEndpoint** objects in memory:

- GET /api/products → maps to GetAll()

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

- GET /api/products/{id} → maps to GetById(int id)

What Is RequestDelegate?

RequestDelegate is a precompiled function that represents the entire process of executing a request for that endpoint. It is like a **mini-pipeline** containing all the steps needed to execute the controller action. Each route has its own dedicated RequestDelegate. When executed, it performs:

- **Model Binding** → Maps query/body/route data to method parameters
- **Validation** → Checks data annotations or validation rules
- **Filter Execution** → Executes filters (authentication, exception, action, result)
- **Controller Activation** → Creates a controller instance via the dependency injection container.
- **Action Invocation** → Calls the match action method of the controller.
- **Response Writing** → Serializes the action's result (JSON, XML, etc.) and writes it to the HTTP response stream.

Step 6: Where the Routing Information Is Stored

After all routes are discovered, ASP.NET Core stores them in an in-memory structure called **EndpointDataSource**. Each subsystems have its own data source:

- **Controllers** → ControllerActionEndpointDataSource
- **Razor Pages** → PageActionEndpointDataSource
- **Minimal APIs** → RouteEndpointDataSource
- **SignalR** → HubEndpointDataSource

These are all combined into a single **CompositeEndpointDataSource** that the **Routing Middleware** uses to perform matching at runtime.

What Happens When a Client Sends a Request in ASP.NET Core Web API

Once the application is started and the routing pipeline is configured, the system is ready to receive requests. Now, when a client sends a request such as: **GET /api/products/5,** the request enters the **Middleware Pipeline**, which is a sequence of components responsible for processing and responding to requests. Let's understand what happens step by step, from the moment the request enters the pipeline to when the response is sent back.

Step 1: Request Enters the Middleware Pipeline

Once the application is started and the app.MapControllers() is configured, and the internal middleware pipeline is ready to process incoming requests. A simplified version of this pipeline looks like this:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
UseRouting()        → Finds which endpoint matches the incoming request.
  ↓
UseAuthorization() → Checks if the user is authorized to access that endpoint.
  ↓
UseEndpoints()      → Executes the matched endpoint (runs the controller action).
```

Every request flows through these middlewares in the exact order they were registered. When the client sends a request such as /api/products/5, it immediately enters this sequence.

Step 2: Routing Middleware Matches the Request

The **purpose of the Routing Middlewar**e is to examine the incoming request, find the matching route template, and attach the corresponding **pre-built endpoint (RouteEndPoint)** to the current request context.

What Happens Internally

1. The middleware reads the incoming request URL path, such as, /api/products/5.
2. It retrieves the collection of **Pre-Built Route Patterns** that were discovered at application startup and stored inside the **EndpointDataSource** (for instance, /api/products/{id}).
3. It compares the request path against these pre-compiled patterns.
4. When a match is found, it:
    - **Extracts the route values** (for example, id = 5).
    - **Selects the corresponding RouteEndpoint** that was already created at startup.
    - **Stores the matched endpoint and route values** inside the current HttpContext.

Internally, this looks like:

1. **context.SetEndpoint(matchedEndpoint);**
2. **context.Request.RouteValues["id"] = 5;**

At this stage:

- No controller logic has executed yet.
- The framework has only determined which action method will handle the request.
- The matched RouteEndpoint and its route values (like id = 5) are stored temporarily in the HttpContext.

This information is later used by other middleware (like Authorization and Endpoint Execution) to actually run the correct action.

Step 3: Authorization and Other Middleware Run

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

After routing determines which endpoint should handle the request, the next stage involves **other middleware components,** such as:

- **Authentication / Authorization Middleware**
- **CORS (Cross-Origin Resource Sharing)**
- **Exception Handling or Logging Middleware**

Each of these middlewares can inspect the HttpContext and access the **Endpoint Metadata** attached to the matched route. For example:

- If the matched controller action has an [Authorize] attribute, the Authorization middleware checks if the user's identity is valid.
- If authorization fails, it can stop further processing and immediately return a 401 Unauthorized response.
- If it succeeds, the request continues to the next step.

This stage ensures that only valid, authorized users reach your controller actions.

Step 4: Endpoint Middleware Executes the Matched Action

Once all middlewares have approved the request, the **Endpoint Middleware** executes the final handler, which is the **Controller Action** matched earlier by the Routing Middleware. This is the stage where your Web API logic actually runs.

The Endpoint Middleware executes the associated compiled **RequestDelegate**, which is a function representing the entire action execution process. This delegate performs several important tasks internally:

1. **Model Binding**: Extracts route values, query strings, headers, or body content and binds them to the action parameters. Example: the {id} in /api/products/5 is bound to the parameter int id.
2. **Validation**: Applies data annotations or custom validation rules on the input model.
3. **Filter Execution**: Runs built-in or custom filters (e.g., authorization filters, action filters, exception filters).
4. **Controller Activation**: Creates a controller instance using dependency injection.
5. **Action Invocation**: Calls the actual controller method (for example, GetById(int id)).
6. **Result Execution**: Serializes the return value (for example, a Product object) into JSON or XML and prepares it for the response.

Step 5: Response Sent Back to the Client

After the controller action completes execution:

1. The result (for example, a product list or message) is **serialized into JSON**.
2. ASP.NET Core writes the response data into the HttpContext.Response.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

3. The middleware pipeline is traversed in **reverse order**, passing through any post-processing middlewares (like logging or exception handlers).
4. Finally, the HTTP response is sent back to the client.

How to See the Registered Endpoints in ASP.NET Core Web API?

You can actually inspect all routes registered in your app. So, please create an empty API Controller named **DiagnosticsController** within the Controllers folder, and then copy-paste the following code. The following code is self-explained, so please read the comment lines for a better understanding.

```
using Microsoft.AspNetCore.Mvc;

namespace RoutingInASPNETCoreWebAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class DiagnosticsController : ControllerBase
    {
        // ASP.NET Core injects all registered EndpointDataSource instances.
        // Each source provides endpoints for controllers, Razor pages, minimal
APIs, etc.
        private readonly IEnumerable<EndpointDataSource> _endpointDataSources;

        public DiagnosticsController(IEnumerable<EndpointDataSource>
endpointDataSources)
        {
            _endpointDataSources = endpointDataSources;
        }

        // Returns detailed runtime information about all discovered endpoints,
        // including route templates, parameters, constraints, defaults, HTTP
methods, and metadata.
        [HttpGet("endpoints")]
        public ActionResult<IEnumerable<EndpointDetailsDTO>>
GetAllEndpointsDetailed()
        {
            var results = new List<EndpointDetailsDTO>();

            // Iterate through each endpoint source registered in DI.
            foreach (var dataSource in _endpointDataSources)
            {
                // Each data source may represent controllers, minimal APIs,
Swagger, etc.
                foreach (var endpoint in dataSource.Endpoints)
                {
                    // Extract Common Metadata
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
                    // RouteNameMetadata and EndpointNameMetadata store
developer-assigned names.
                    var routeName =
endpoint.Metadata.GetMetadata<IRouteNameMetadata>()?.RouteName;
                    var endpointName =
endpoint.Metadata.GetMetadata<EndpointNameMetadata>()?.EndpointName;

                    // HttpMethodMetadata contains information about allowed HTTP
verbs (GET, POST, etc.)
                    var httpMethodMetadata =
endpoint.Metadata.GetMetadata<HttpMethodMetadata>();
                    var httpMethods = httpMethodMetadata?.HttpMethods?.ToArray()
?? Array.Empty<string>();

                    // Attribute metadata: collects attribute types applied to
controller actions (e.g., [HttpGet], [Authorize]).
                    var attributeTypeNames = endpoint.Metadata
                        .OfType<Attribute>()
                        .Select(a => a.GetType().FullName ?? a.GetType().Name)
                        .ToArray();

                    // Initialize DTO with base details
                    var details = new EndpointDetailsDTO
                    {
                        DisplayName = endpoint.DisplayName, // human-readable
identifier
                        EndpointType = endpoint.GetType().FullName ??
endpoint.GetType().Name, // RouteEndpoint, Endpoint, etc.
                        IsRouteEndpoint = endpoint is RouteEndpoint, // true if
it supports a route pattern
                        RouteName = routeName,
                        EndpointName = endpointName,
                        HttpMethods = httpMethods,
                        AttributeMetadataTypeNames = attributeTypeNames
                    };

                    // Extract Route-Specific Data (if this is a RouteEndpoint)
                    if (endpoint is RouteEndpoint routeEndpoint)
                    {
                        var pattern = routeEndpoint.RoutePattern;

                        // Raw route pattern text (e.g.,
"api/products/{id:int:min(1)=1}")
                        details.Pattern = pattern.RawText;

                        // Route Defaults
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
                        // Example: {id=1} produces "id" -> "1"
                        if (pattern.Defaults.Count > 0)
                        {
                            details.PatternDefaults = pattern.Defaults
                                .ToDictionary(kv => kv.Key, kv =>
kv.Value?.ToString());
                        }

                        // Route Parameters
                        // Each parameter may have:
                        //   - a name (e.g., "id")
                        //   - an optional flag (e.g., "{id?}")
                        //   - a default value (e.g., "{id=1}")
                        //   - constraints/policies (e.g., ":int", ":min(1)")
                        if (pattern.Parameters.Count > 0)
                        {
                            details.Parameters = pattern.Parameters
                                .Select(p => new RouteParameterDetailsDTO
                                {
                                    Name = p.Name,
                                    IsOptional = p.IsOptional,
                                    DefaultValue = p.Default?.ToString(),

                                    // Each ParameterPolicyReference contains a
"Content" object that describes the constraint.
                                    // Converting it to string gives readable
names like "int", "min(1)", etc.
                                    Policies = p.ParameterPolicies
                                        .Select(pol => pol.Content?.ToString() ??
string.Empty)
                                        .ToArray()
                                })
                                .ToArray();
                        }
                    }

                    // Add to results
                    results.Add(details);
                }
            }

            // Return as JSON
            return Ok(results);
        }
    }

    //DTO DEFINITIONS
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    // Represents detailed information about a single endpoint.
    public sealed class EndpointDetailsDTO
    {
        // ---------- General ----------
        public string? DisplayName { get; set; }            // e.g.,
"ProductsController.GetById (RoutingInASPNETCoreWebAPI)"
        public string? EndpointType { get; set; }           // e.g.,
"Microsoft.AspNetCore.Routing.RouteEndpoint"
        public bool IsRouteEndpoint { get; set; }           // true if it's a
RouteEndpoint (as opposed to a non-routed endpoint like Swagger)

        // ---------- Route Endpoint Specific ----------
        public string? Pattern { get; set; }                // Full route
pattern string (e.g., "api/products/{id:int:min(1)=1}")
        public Dictionary<string, string?>? PatternDefaults { get; set; } //
Route-level or parameter-level default values
        public RouteParameterDetailsDTO[]? Parameters { get; set; }      // All
parameters in the pattern

        // ---------- Metadata ----------
        public string? RouteName { get; set; }              // From
[HttpGet(Name = "...")] or similar
        public string? EndpointName { get; set; }           // From
EndpointNameMetadata (used by minimal APIs)
        public string[] HttpMethods { get; set; } = Array.Empty<string>(); //
Allowed HTTP verbs (GET, POST, PUT, etc.)
        public string[] AttributeMetadataTypeNames { get; set; } =
Array.Empty<string>(); // Attribute metadata attached to this endpoint
    }

    // Represents details about a single route parameter, including constraints
and defaults.
    public sealed class RouteParameterDetailsDTO
    {
        public string Name { get; set; } = string.Empty;     // Parameter name
(e.g., "id")
        public bool IsOptional { get; set; }                // True if the
parameter is optional ({id?})
        public string? DefaultValue { get; set; }           // Default value if
defined ({id=1})
        public string[] Policies { get; set; } = Array.Empty<string>(); //
Parameter constraints/policies (e.g., "int", "min(1)")
    }
}
```

Now, run the application and access the **GET /api/diagnostics/endpoints** endpoint, and you should see the list of all registered routing endpoints.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Middleware Ordering in ASP.NET Core's Minimal Hosting Model:

In ASP.NET Core's **Minimal Hosting Model** (introduced in .NET 6), some essential middleware components, such as **Routing** and **Endpoint Execution**, are automatically added by the framework when you use mapping methods like app.MapControllers() or app.MapGet(). This means you no longer need to explicitly call app.UseRouting() or app.UseEndpoints() for routing to function correctly.

However, the **order of middleware registration** in the request pipeline still determines how each request is processed. The minimal hosting model does not remove the importance of ordering. It simply ensures that internally required middleware, such as routing, is automatically inserted in the correct position when you use endpoint-mapping methods. For example, even if you write your configuration like this:

- **UseAuthorization()**
- **MapControllers()**

The framework internally arranges the effective runtime order as:

- **UseRouting()**
- **UseAuthorization()**
- **EndpointMiddleware**

This ensures that the routing middleware always runs **before** authorization, allowing the system to identify which endpoint is being accessed before applying any authorization checks. In short, the minimal hosting model doesn't override ordering rules; it just **inserts required middleware automatically** in the correct sequence to simplify configuration.

Conclusion

Routing in ASP.NET Core Web API is a powerful and well-organized system that maps incoming HTTP requests to the appropriate controller actions. During application startup, all routes are discovered, compiled into RouteEndpoint objects, and stored in memory for fast matching.

When a request arrives, the Routing Middleware simply matches it to one of these pre-built endpoints and attaches it to the HttpContext, while later middleware like Authorization and Endpoint Execution handle security checks and action invocation. This design makes routing both efficient and flexible, ensuring that every request is processed quickly, accurately, and according to the defined route patterns and constraints.

In the next article, I will discuss the different **Controller Action Method Return Types in ASP.NET Core Web API** Applications with Examples. In this article, I try to explain Route Constraints in ASP.NET Core Web API applications with examples. And I hope you enjoy this article.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF